CS 225

Week 5: Queues and Iterators

March 16-20, 2026 V. Kindratenko

Queue Implementation #1

Queue.h

```
4  template <class QE>
5  class Queue {
6    public:
7
8
9
10
11   private:
12
13
14
15
16 };
```

A queue is a _____ data structure!

What type of implementation if this Queue?

How is the data stored in this Queue?

Which pointer is entry and which pointer is exit?

What is the running time of enqueue()?

What is the running time of dequeue()?

Queue Implementation #2

Queue.h

```
4  template <class QE>
5  class Queue {
6    public:
7      void enqueue(QE e);
8      QE dequeue();
9      bool isEmpty();
10
11   private:
12     QE *items;
13     unsigned capacity_;
14     unsigned count_;
15
16
17
18 };
```

What type of implementation if this Queue?

How is the data stored in this Queue?

Example 1

```
Queue<int> q;
q.enqueue(3);
q.enqueue(8);
q.enqueue(4);
q.dequeue();
q.enqueue(7);
q.dequeue();
q.dequeue();
q.enqueue(2);
q.enqueue(1);
q.enqueue(3);
q.enqueue(5);
q.dequeue();
q.enqueue(9);
```

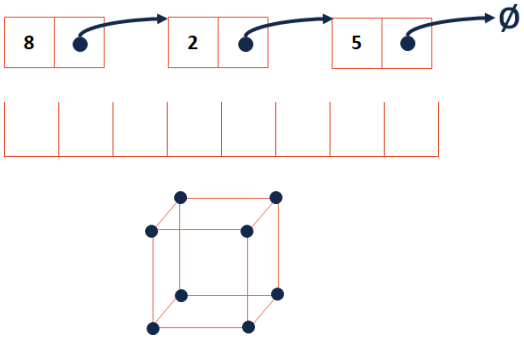
Example 2

```
Queue<char> q;
q.enqueue(m);
q.enqueue(o);
q.enqueue(n);
…
q.enqueue(d);
q.enqueue(a);
q.enqueue(y);
q.enqueue(i);
q.enqueue(s);
q.dequeue();
q.enqueue(h);
q.enqueue(a);
```

Iterators

Purpose:

…on what data structures?



Operators:

Iterator Types:

stlList.cpp

```
 1  #include <list>
 2  #include <string>
 3  #include <iostream>
 4
 5  struct Animal {
 6    std::string name, food;
 7    bool big;
 8    Animal(std::string name = "blob", std::string food = "you",
    bool big = true) :
 9      name(name), food(food), big(big) { /* none */ }
10  }
11
12  int main() {
13    Animal g("giraffe", "leaves", true),
             p("penguin", "fish", false), b("bear");
14    std::list<Animal> zoo;
15
16    zoo.push_back(g);
17    zoo.push_back(p);   // std::list's insertAtEnd
18    zoo.push_back(b);
19
20    for ( std::list<Animal>::iterator it = zoo.begin();
                                          it != zoo.end(); it++ )
21  {
22      std::cout << (*it).name << " " << (*it).food << std::endl;
23    }
24
25    return 0;
    }
```

CS 225

#Week 5: Functors

March 16-20, 2026 V. Kindratenko

Queue Iterator:

QueueIter.h

```
 4  template <class QE>
 5  class Queue {
 6    public:
 7
 8
 9
10
11
12
13
14
15
16    private:
17
18
19
20  };
```

Where does the iterator go?

What additional functions exist with an iterator? / Where do they go?

1.

2.

3.

4.

stlList.cpp

```
 1  #include <list>
 2  #include <string>
 3  #include <iostream>
 4
 5  struct Animal {
 6    std::string name, food;
 7    bool big;
 8    Animal(std::string name = "blob", std::string food = "you",
    bool big = true) :
 9      name(name), food(food), big(big) { /* none */ }
10  }
11
12  int main() {
13    Animal g("giraffe", "leaves", true),
             p("penguin", "fish", false), b("bear");
14    std::list<Animal> zoo;
15
16    zoo.push_back(g);
17    zoo.push_back(p);   // std::list's insertAtEnd
18    zoo.push_back(b);
19
20    for ( std::list<Animal>::iterator it = zoo.begin();
                                          it != zoo.end(); it++ )
21  {
22      std::cout << (*it).name << " " << (*it).food << std::endl;
23    }
24
25    return 0;
  }
```

Function Objects ("Functors")

Purpose:

Example:

How is this accomplished?

Doubler.h/.cpp

```
4  class Doubler {
5    public:
6      int operator()(int value);
7  };

3  int Doubler::operator()(int value) {
4    return value * 2;
5  }
```

What does Doubler do?

How do we know Doubler is a functor?

What can functors do that an ordinary function cannot do?

---

example1.cpp

```
1  template<class Value, class Modifier>
2  Value modify(Value value, Modifier modifier) {
3     return modifier(value);
4  }
…  /* … */
11 Doubler d;
12 Tripler t;
13 int value = 4;
14 modify<int, Doubler>(value, d);
15 modify<int, Tripler>(value, t);
```

How are functors used?

When is the modifier function valid?

example2.cpp

```
1  template<class Iter, class Formatter>
2  void print(Iter first, Iter second, Formatter printer) {
3    while ( first != second ) {
4      printer( *first );
5      first++;
6
7    }
   }
```

This is a function called _____ whose inputs are two

_____ and a _____.

This function appears to:

Queue Iterator:

QueueIter.h

```
4  template <class QE>
5  class Queue {
6    public:
7      class QueueIterator :
       public std::iterator<std::bidirectional_iterator_tag, T> {
8        public:
9          QueueIterator(unsigned index);
10         QueueIterator& operator++();
11         bool operator==(const QueueIterator &other);
12         bool operator!=(const QueueIterator &other);
13         QE& operator*();
14         QE* operator->();
15       private:
16         int location_;
17
18     };
19   /* ... */
20   private:
21     QE* arr_; unsigned capacity_, count_, entry_, exit_;
22 };
```

Does an instance of a **`QueueIterator`** have access to the **`Queue`** **`arr_`**?

Two big takeaways:

1.

2.

CS 225

#Week 5: Trees

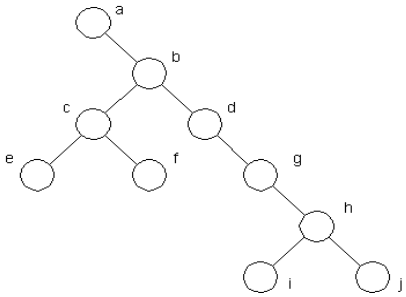
March 16-20, 2026 V. Kindratenko

Trees!

"The most important non-linear data structure in computer science."
- David Knuth, The Art of Programming, Vol. 1

A tree is:

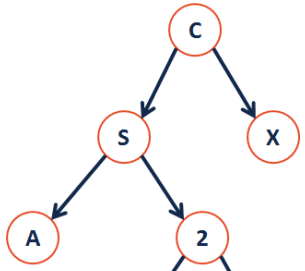
We are going to start with a specific type of tree:



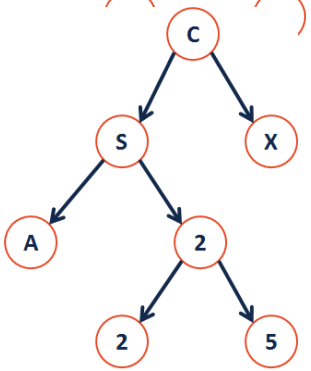
- What's the longest "word" you can make using the **vertex** labels in the tree (repeats allowed)?
- Find an **edge** that is not on the longest **path** in the tree. Give that edge a reasonable name.
- One of the vertices is called the **root** of the tree. Which one?
- Make a "word" containing the names of the vertices that have a **parent** but no **sibling**.
- How many parents does each vertex have?
- Which vertex has the fewest **children**?
- Which vertex has the most **ancestors**?
- Which vertex has the most **descendants**?
- List all the vertices is b's left **subtree**.
- List all the **leaves** in the tree.

Definition: Binary Tree

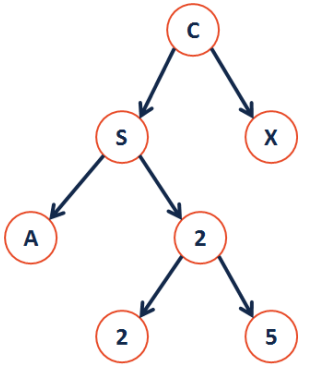
A *binary tree* **T** is either:



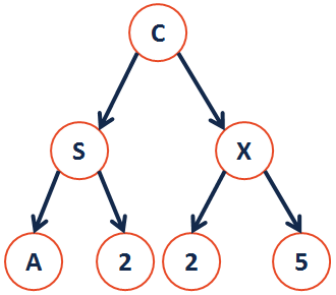
Tree Property: Tree Height



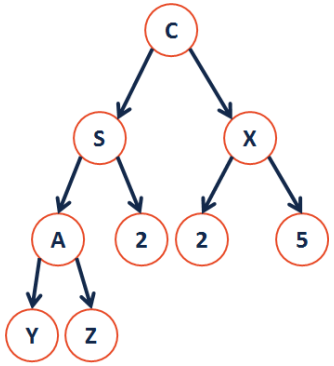
Tree Property: Full



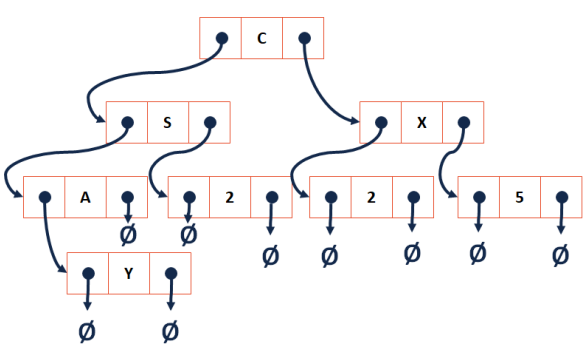
Tree Property: Perfect

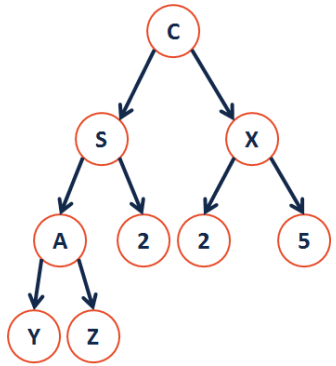


Tree Property: Complete




```
 9
10
11
12
13
14
15
16    private:
17
18
19
20
21
22
23
24
25
26  };
27  #endif
```





Trees are nothing new:

Tree Class

BinaryTree.h

```
1  #ifndef BINARYTREE_H
2  #define BINARYTREE_H
3
4  template <class T>
5  class BinaryTree {
6    public:
7
8
```

Theorem: If there are n data items in our representation of a binary tree, then there are _____ NULL pointers.

CS 225 – Things To Be Doing:

1. MP3: up to +7 for submitting by March 23
2. lab_quacks due this Sunday, March 22 (11:59pm)